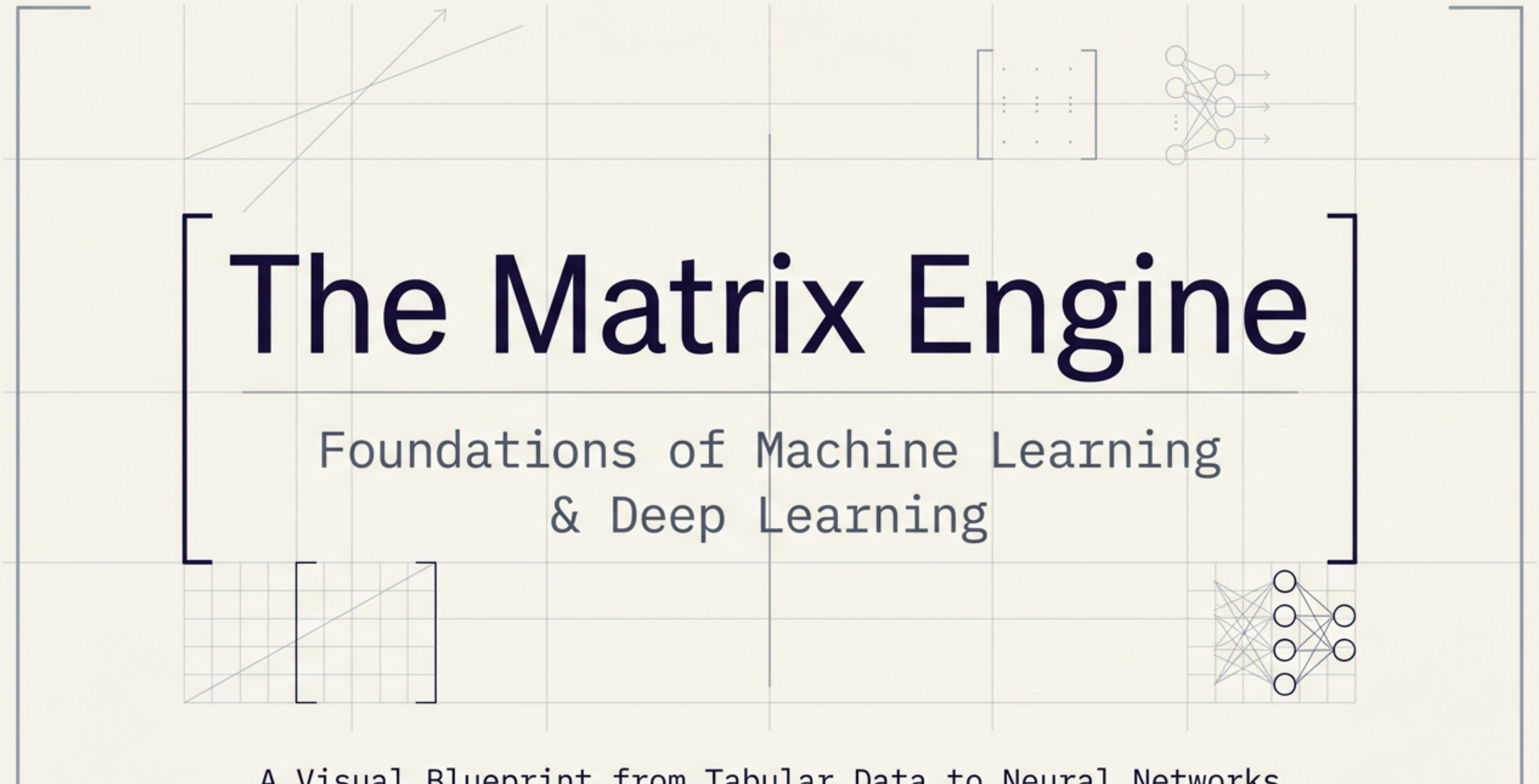




The Matrix Engine

Foundations of Machine Learning
& Deep Learning

A Visual Blueprint from Tabular Data to Neural Networks

Why Do We Need Matrices?

Most real-world datasets are tabular. A matrix perfectly mirrors this structure, allowing computers to process entire datasets as a single mathematical object.

Data Points / Individual Observations		A		B	
		Features & Targets			
		Study Hours (Input Feature)		Marks (Target Output)	
	1	3		5	
	2	5		20	
	3	6		35	
		Study Hours (Feature 1)	Play Hours (Feature 2)	Marks (Target Output)	
	4	4	2	25	
	5	2	5	10	

Mathematical
Translation



Example 1 (Single Feature)

$$\begin{bmatrix} 3 & 5 \\ 5 & 20 \\ 6 & 35 \end{bmatrix}$$

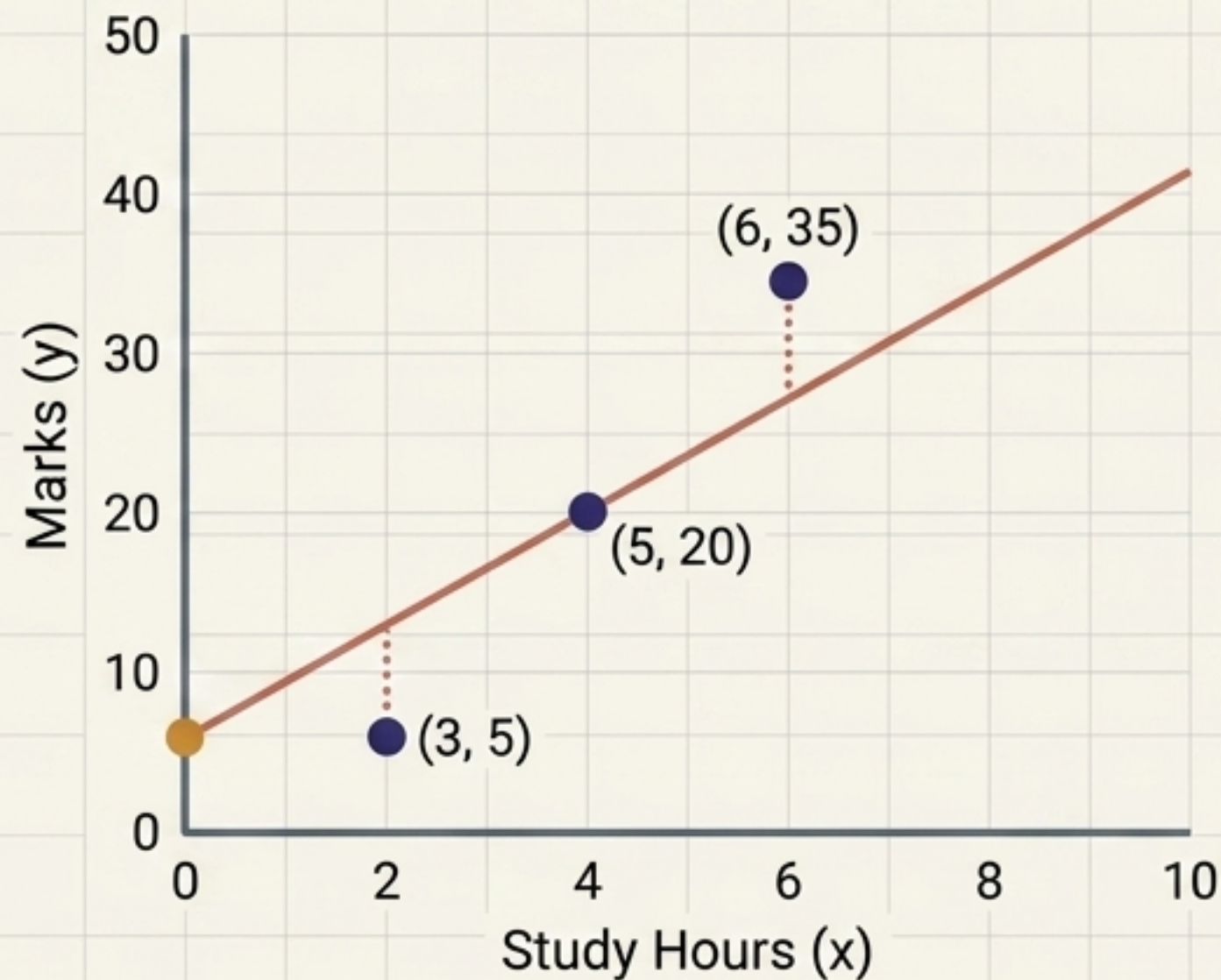
Example 2 (Multi-Feature)

$$\begin{bmatrix} 4 & 2 & 25 \\ 2 & 5 & 10 \end{bmatrix}$$

The Core ML Logic

Linear Regression in 2D

The model attempts to draw a best-fit line through data points. By adjusting the weight and bias, it mathematically aligns with the data to predict unseen inputs.



$$\text{Prediction} = (\text{Weight} \times \text{Feature}) + \text{Bias}$$

$$y = (w \times x) + b$$

y (Prediction)

w (Slope/Terracotta)

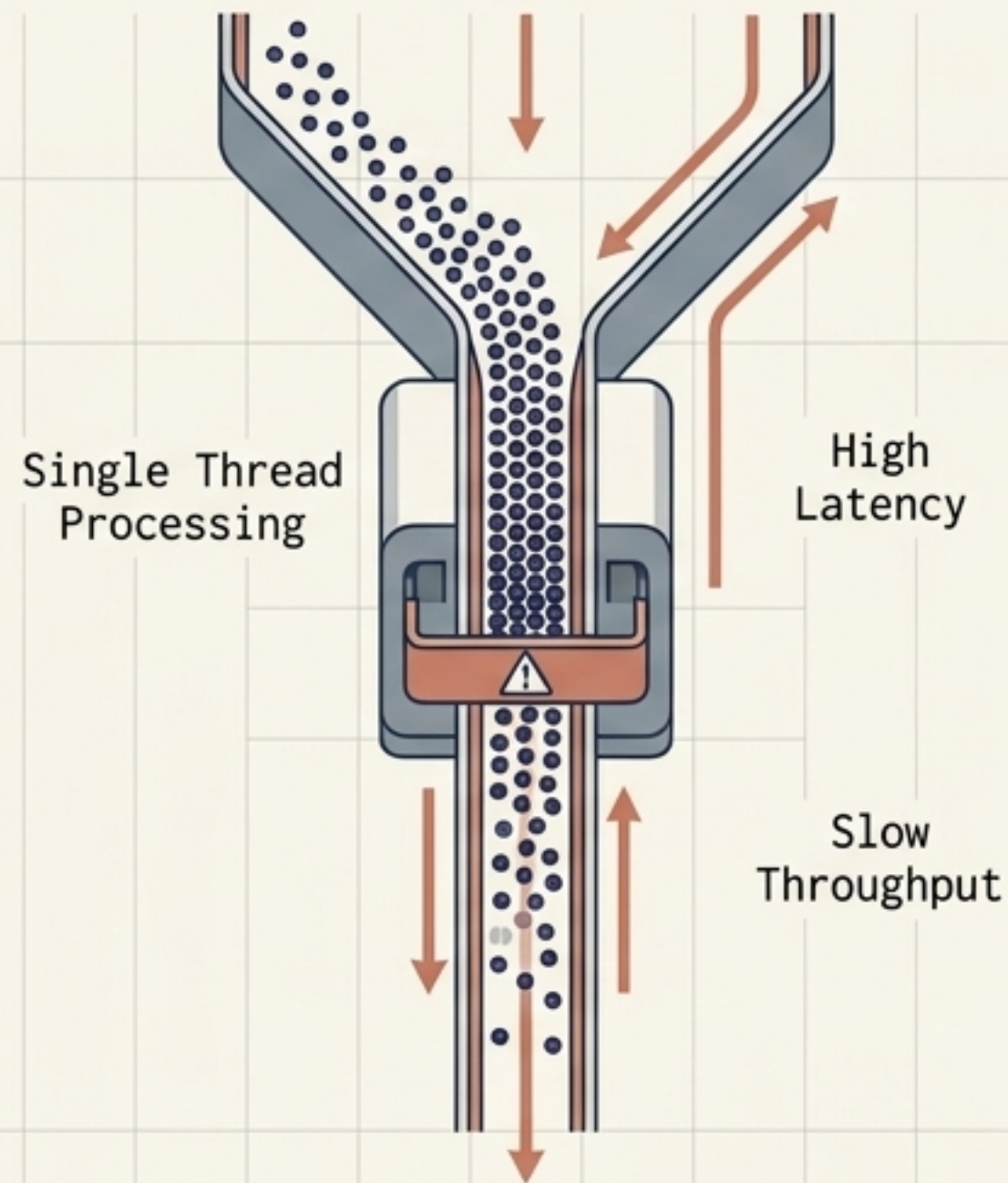
x (Data/Indigo)

b (Intercept/Ochre)

16:9 The Calculation Problem: Scaling Up

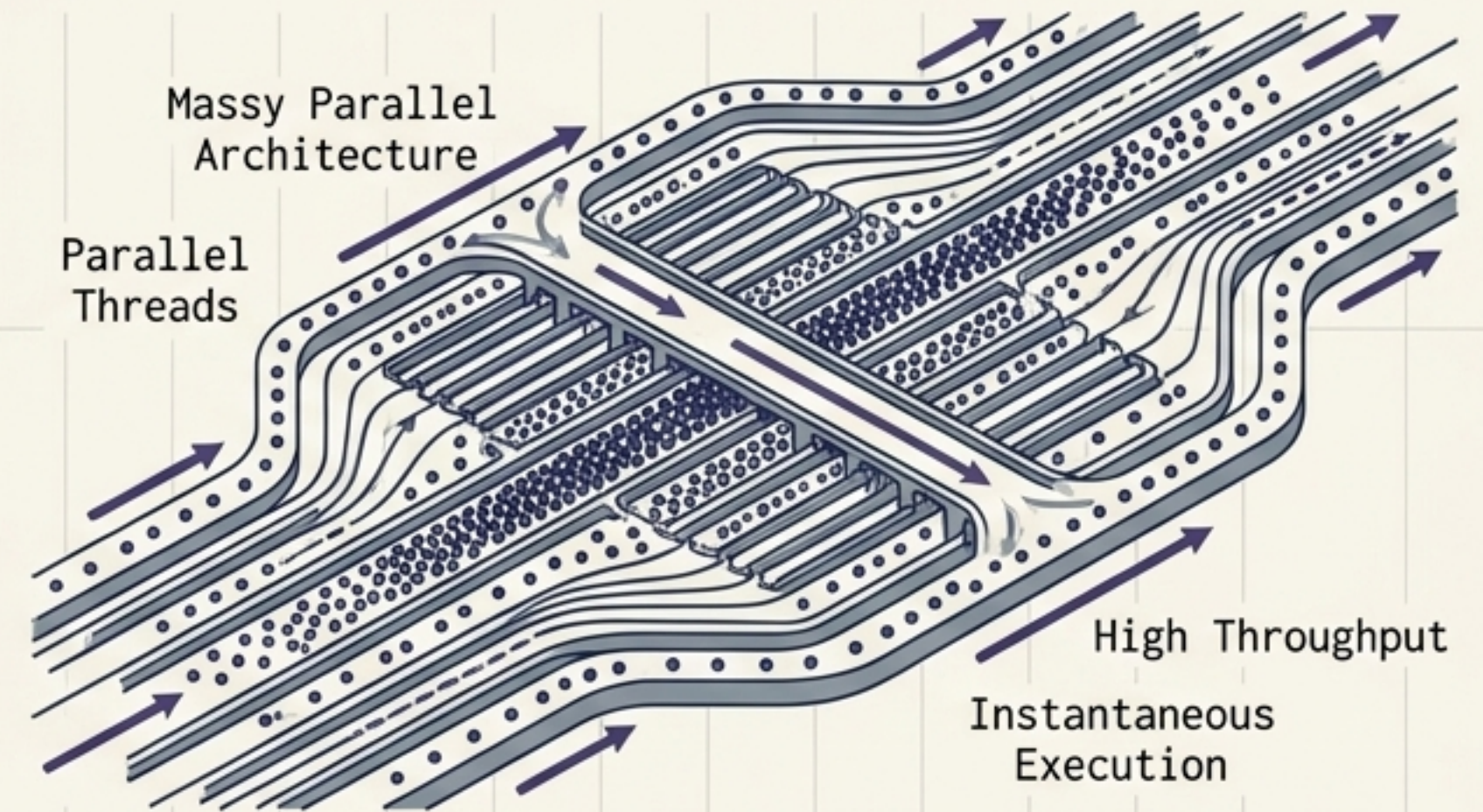
Standard Loops (CPU Bottleneck)

Calculating $y = (w \times x) + b$ one by one for millions of rows is incredibly slow.



Matrix Math (SIMD & GPU Parallelism)

Grouping weights and features into matrices unlocks parallel computing, turning days of computation into seconds.



Hardware Advantages:

1. **SIMD (Single Instruction, Multiple Data):** CPU processes multiple points simultaneously.
2. **GPU Parallelism:** Executes thousands of matrix operations at the exact same time.

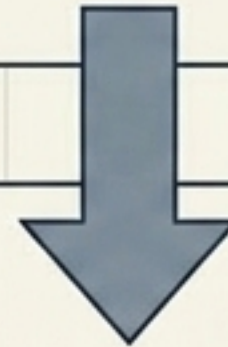
The Hierarchy of Dimensions

Scalar (0D)

A single value with no direction.

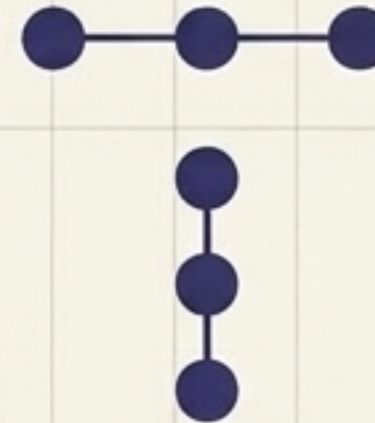


15

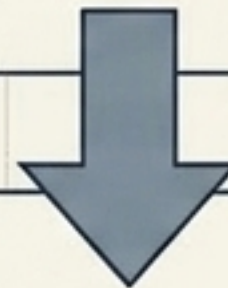


Vector (1D)

A 1-dimensional array. Can be a horizontal Row or vertical Column.

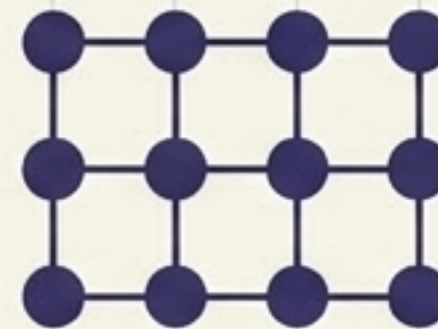


[10, 20, 30]



Matrix (2D)

A 2-dimensional grid of numbers, built by stacking multiple vectors together.



$\begin{bmatrix} \cdot & \dots & \cdot \\ \cdot & \dots & \cdot \\ \cdot & \dots & \cdot \end{bmatrix}$

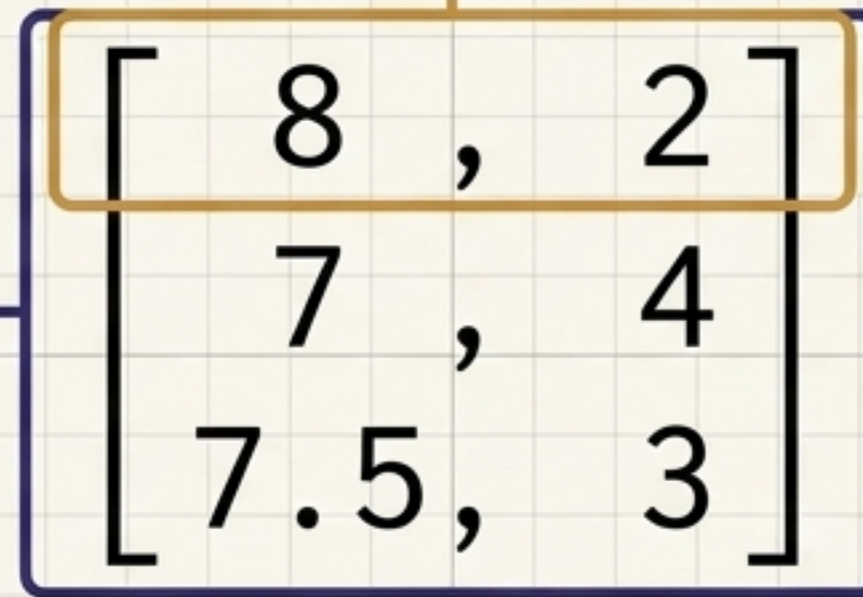


Understanding Matrix Shape

The Golden Rule: Shape is always defined as Rows $\times$ Columns.

Columns = 2 (Independent Variables / Features)

Rows = 3
(Unique Data Points / Samples)



8	2
7	4
7.5	3

Dataset Blueprint

Shape: 3 $\times$ 2 Matrix

Total Elements: 3 $\times$ 2 = 6

Expert Insight: If a dataset is shaped 200,000 $\times$ 3, its exact blueprint is instantly known: 200,000 unique samples, each described by 3 features.

Matrix Classifications in ML

Row Vector ($1 \times n$)

$[10, 20, 30]$

ML Context: A single data point fed into a model.

Column Vector ($n \times 1$)

$\begin{bmatrix} 10 \\ 30 \\ 40 \end{bmatrix}$

ML Context: The target output column or final prediction.

Square Matrix ($n \times n$)

$\begin{bmatrix} 4 & 2 & 1 \\ 5 & 9 & 3 \\ 0 & 6 & 8 \end{bmatrix}$

Primary diagonal runs top-left to bottom-right.
Example: Covariance matrices in PCA.

Identity Matrix (I)

$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Rule: $A \times I = A$. Acts like multiplying a number by 1.

Transpose Matrix (A^T)

$$A = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \longrightarrow A^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Flipping over the diagonal. Rows become columns, and columns become rows.

Standard Operations: The Exact Shape Rule

For standard addition and subtraction, both matrices **MUST** have the exact same shape. The math happens element-wise.

Matrix A Matrix B Result Matrix

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{bmatrix}$$

The diagram illustrates the addition of two 2x3 matrices. Matrix A is $\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$ and Matrix B is $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$. The result matrix is $\begin{bmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{bmatrix}$. Dashed orange arrows point from the top row of Matrix A to the top row of the Result Matrix, and from the bottom row of Matrix A to the bottom row of the Result Matrix. Dashed blue arrows point from the first column of Matrix A to the first column of the Result Matrix, from the second column of Matrix A to the second column of the Result Matrix, and from the third column of Matrix A to the third column of the Result Matrix.

The **Twist**: What happens in ML when the shapes don't match?

The Magic of Broadcasting

ML libraries automatically "stretch" smaller arrays to match larger ones for element-wise operations.

Scalar (0D to 2D)

Matrix A

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} + 10$$

Stretches to fit

$$\text{Matrix A} + \begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Row Vector (1D to 2D)

Matrix A

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} + [1, 2, 3]$$

Stretches downwards

$$\text{Matrix A} + \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix}$$

Column Vector (1D to 2D)

Matrix A

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Stretches horizontally

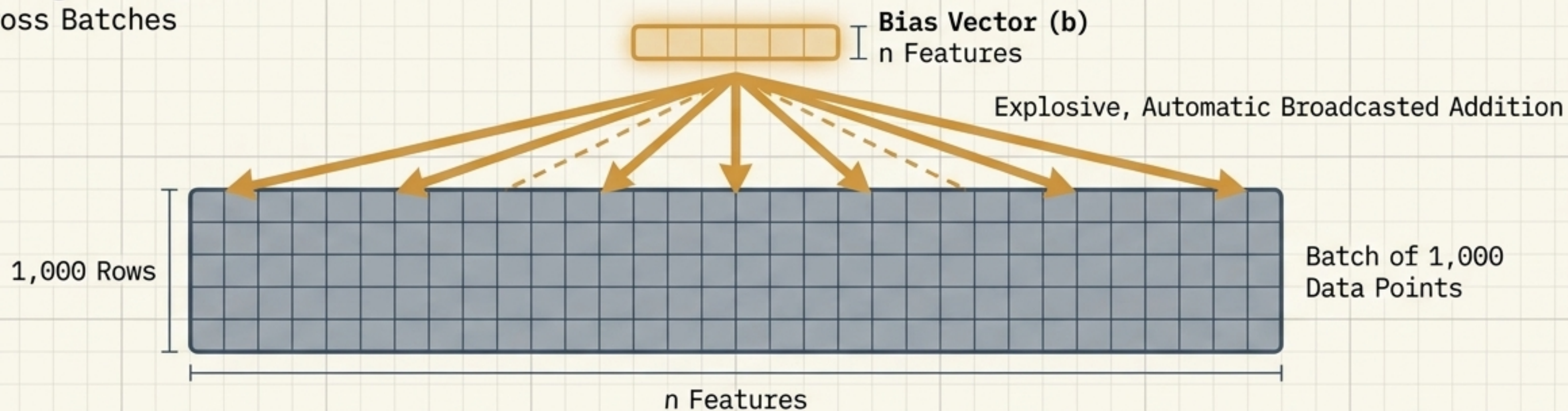
$$\text{Matrix A} + \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix}$$

⚠ Failure Condition: Inner dimensions must align. A 1×4 vector cannot broadcast into a 2×3 matrix.

Why Broadcasting Powers AI

1. Adding Bias (+b)

Across Batches



2. Gradient Descent

Parameter Updates

$$W_{new} = W_{old} - (\text{learning_rate} \times \text{gradient})$$

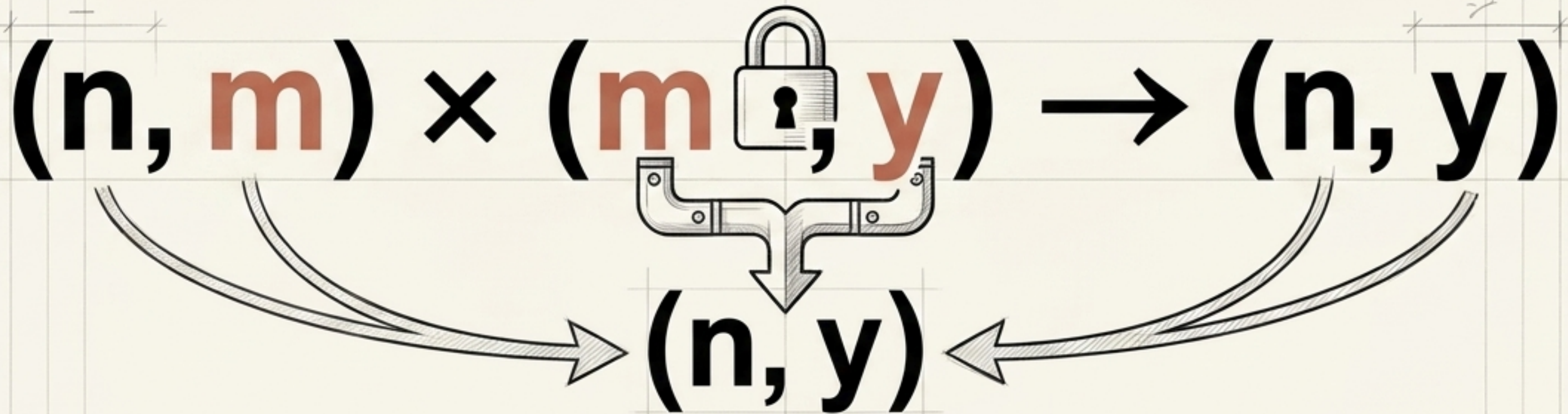
Scalar multiplication and matrix subtraction are broadcast to update millions of weights in a single, parallelized sweep without loops.

Matrix Multiplication: The Engine

Unlike addition, this is NOT element-by-element. It uses the Dot Product of rows and columns.

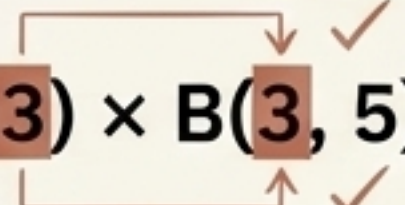
The Golden Rule of Dimensions

Inner dimensions MUST be identical



Valid Match

$A(2, 3) \times B(3, 5) \rightarrow$ Result shape: $(2, 5)$

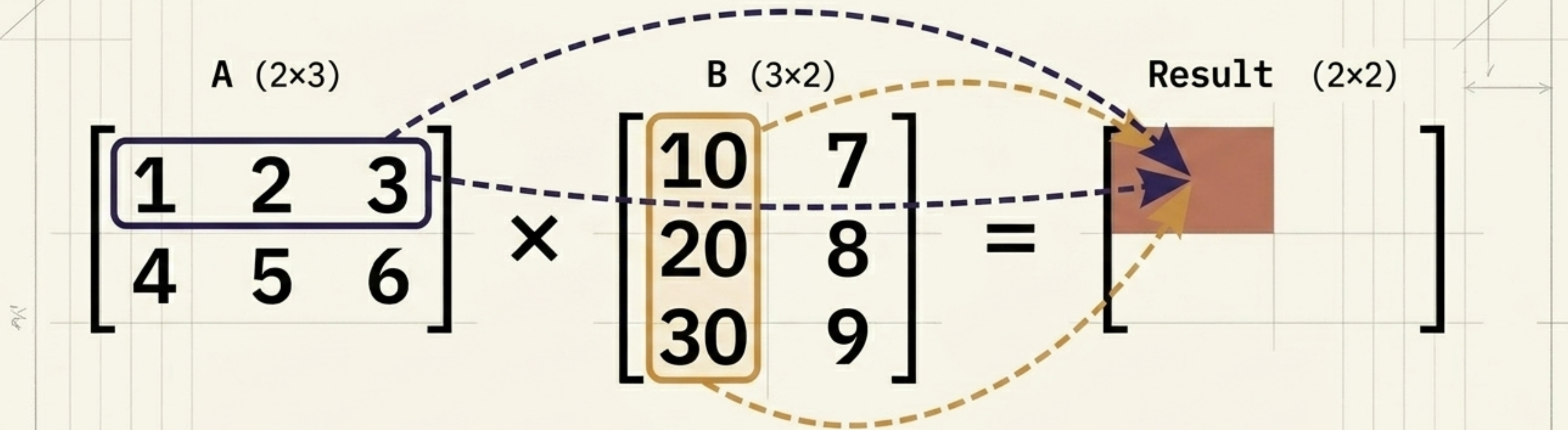


Invalid Crash

$A(2, 3) \times B(5, 7) \rightarrow$ Operation Crashes



Step-by-Step: The Row $\times$ Column Operation



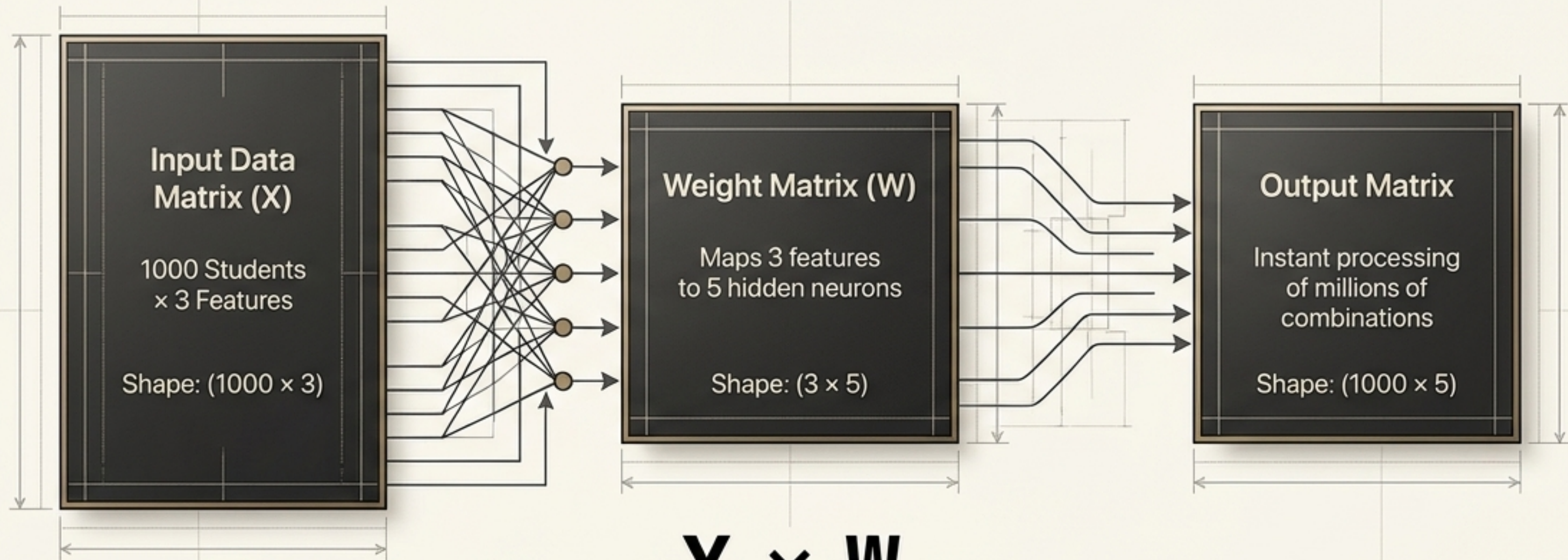
Top-Left Cell Calculation:

$$(1 \times 10) + (2 \times 20) + (3 \times 30) = 140$$

Final 2x2 Result: $\begin{bmatrix} 140 & 50 \\ 320 & 122 \end{bmatrix}$

Synthesis: The Neural Network Forward Pass

Matrix multiplication allows ML to process entire datasets instantly without loops.

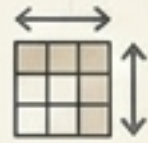


$$\mathbf{X} \times \mathbf{W}$$
$$(1000 \times 3) \times (3 \times 5) \rightarrow (1000 \times 5)$$

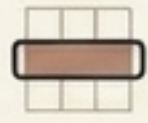
Bonus Context: PCA
Need to reduce 100 features to 2 for graph visualization?
Multiply by a crafted Transformation Matrix!

The Matrix Engine: Key Takeaways

Fundamentals



Shape =
Rows $\times$ Columns.



Row = Unique
Data Point.

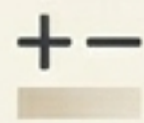


Column =
Independent Feature.

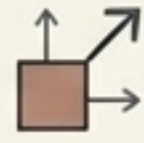


Identity Matrix (I):
 $A \times I = A$.

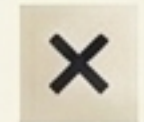
The Rules



Addition/Subtraction:
Exact same shapes
required.



Broadcasting:
Automatically stretches
shapes for biases (+b)
and updates.

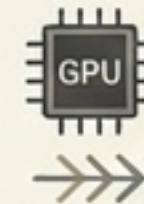


Multiplication:
Inner dimensions
MUST match.
 $(n,m) \times (m,y) \rightarrow (n,y)$.

The "Why"



Why Matrices?
They perfectly mirror
Tabular Data.



Why Matrix Math?
Unlocks GPU parallel
processing (SIMD).



Result: Turns days
of loops into
milliseconds of
Deep Learning.